Laboratório 9: Desenvolvimento Seguro

Modelagem de Sistemas · Prof. Dr. Rodrigo Juliani · Aluno: Pedro Brito

Exercício 9.1: Modelagem de Ameaças e CIA

Portal de uma clínica: pacientes veem resultados de exames e médicos prescrevem receitas.

Pilar	Ameaça potencial	Medida preventiva
Confidencialidade	Um paciente logado troca o número na URL (`/exames/123` → `/exames/124`) e abre o exame de outra pessoa.	Login individual e, no servidor, checagem de dono em toda requisição.
Integridade	Alguém altera medicamento ou dose de uma receita já emitida, no banco de dados ou no arquivo.	Assinatura digital do médico e receita imutável depois de emitida.
Disponibilidade	Um ataque DoS no horário de pico derruba o portal.	*Rate limiting* e proteção anti-DDoS, com plano de contingência.

Confidencialidade: como garantir que apenas o paciente veja seu exame? O login diz *quem* é o usuário, mas não basta: o servidor também precisa verificar, a cada acesso, se aquele exame pertence a ele (`exame.paciente_id == usuario_logado.id`). Se não pertencer, nega o acesso. A autorização nunca pode depender só de esconder o link na tela. HTTPS protege os dados em trânsito, e a criptografia protege os dados armazenados.

Integridade: como garantir que a receita não foi alterada após ser emitida? O sistema calcula o hash do conteúdo da receita e o médico o assina com sua chave privada (por exemplo, um certificado ICP-Brasil). Qualquer mudança faz a verificação da assinatura falhar. Um hash sozinho não basta, porque quem altera o conteúdo pode recalcular o hash; quem não tem a chave do médico não consegue refazer a assinatura. A receita emitida não é editada: uma correção gera uma nova versão, registrada em log de auditoria.

Disponibilidade: o que acontece se o servidor sofrer um DoS no horário de pico? O portal fica lento ou fora do ar. Pacientes não acessam exames, médicos não conseguem prescrever, as filas crescem e o atendimento atrasa, o que pode trazer risco à saúde. Para prevenir, o sistema limita requisições por IP e usuário (*rate limiting*), usa WAF/CDN com proteção anti-DDoS e mantém servidores redundantes. Para o caso de o ataque passar, a clínica precisa de um plano de contingência, como receita em papel e exames entregues por outro canal.

Exercício 9.2: Identificação de Vulnerabilidades

Categorias da OWASP Top 10:2025.

a) `delete_user`

```
@app.route('/delete_user/<user_id>')
def delete_user(user_id):
    db.execute(f"DELETE FROM users WHERE id = {user_id}")
    return "Usuário deletado"
```

Categoria: A01:2025 Broken Access Control, junto com **A05:2025 Injection**. O trecho tem as duas falhas, e corrigir uma não resolve a outra.

- **A01:** a rota não exige login nem perfil de administrador. Qualquer pessoa que acesse `/delete_user/5` apaga o usuário 5. Como a rota aceita GET, um simples link malicioso ou imagem em outra página também pode disparar a exclusão.
- **A05:** o `user_id` entra direto no SQL. O valor `1 OR 1=1` vira `DELETE FROM users WHERE id = 1 OR 1=1`, que **apaga a tabela inteira**.
- **Risco:** perda de dados e indisponibilidade do sistema para todos os usuários.
- **Correção:** exigir autenticação e perfil de administrador, aceitar só `POST`/`DELETE` com proteção CSRF, usar `<int:user_id>` e consulta parametrizada.

b) `config`

```
config = {
    "host": "localhost",
    "user": "admin",
    "password": "password123",  # Senha padrão do sistema
    "database": "clinica_db"
}
```

Categoria: A02:2025 Security Misconfiguration. A OWASP cita em A02 contas padrão com senhas que continuam ativas e inalteradas. A credencial escrita no código também se enquadra em **A07:2025 Authentication Failures**, que mapeia os CWE-798 (*Hard-coded Credentials*) e CWE-1392 (*Default Credentials*).

- **Risco:** `admin`/`password123` é facilmente adivinhada e fica exposta a quem tiver acesso ao código ou ao histórico do Git. Com ela, um invasor lê, altera ou apaga todo o `clinica_db`, com dados de saúde dos pacientes.
- **Correção:** trocar a senha, guardá-la em variável de ambiente ou cofre de segredos, usar um usuário de banco com o mínimo de privilégios e remover o segredo do histórico do repositório.

c) `get_profile`

```
# Função para carregar perfil de usuário
def get_profile(request):
    user_id = request.args.get('id')
    # O sistema busca os dados sem verificar se o cookie de sessão pertence...
    return db.query(f"SELECT * FROM profiles WHERE id = {user_id}")
```

Categoria: A01:2025 Broken Access Control, no caso conhecido como **IDOR** (*Insecure Direct Object Reference*). Como em (a), a f-string também abre espaço para **A05:2025 Injection**.

- **Risco:** basta mudar `?id=` para ler o perfil de qualquer usuário. Um script pode percorrer os IDs e copiar todos os perfis, gerando vazamento em massa de dados pessoais e incidente de LGPD. O `SELECT *` piora o problema, pois devolve todas as colunas, inclusive as que a tela não usa.
- **Correção:** obter o usuário da sessão no servidor, negar acesso a perfis de outras pessoas, usar consulta parametrizada e selecionar só as colunas necessárias.

Exercício 9.3: Refatoração para Segurança

Código completo e testes em `exercicio 9 3/`.

```
import re
import sqlite3
from contextlib import closing

CAMINHO_BANCO = "usuarios.db"
TAMANHO_MAXIMO_EMAIL = 254
PADRAO_EMAIL = re.compile(
    r"[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+(\.[A-Za-z0-9-]+)*\.[A-Za-z]{2,}"
)


def validar_email(email_digitado):
    """Devolve o e-mail sem espaços nas pontas ou levanta ValueError se for inválido."""
    if not isinstance(email_digitado, str):
        raise ValueError("E-mail inválido.")

    email = email_digitado.strip()
    # fullmatch exige que a string inteira siga o padrão, do início ao fim.
    if len(email) > TAMANHO_MAXIMO_EMAIL or not PADRAO_EMAIL.fullmatch(email):
        raise ValueError("E-mail inválido.")

    return email
```

```
def busca_usuario_por_email(email_digitado, caminho_banco=CAMINHO_BANCO):
    """Devolve (nome, email) do usuário encontrado ou None."""
    email = validar_email(email_digitado)

    # closing() garante que a conexão seja fechada mesmo se ocorrer um erro.
    with closing(sqlite3.connect(caminho_banco)) as conexao:
        # O "?" envia o e-mail como DADO: ele nunca vira parte do comando SQL.
        cursor = conexao.execute(
            "SELECT nome, email FROM usuarios WHERE email = ?",
            (email,),
        )
        return cursor.fetchone()
```

Mudança	Por quê
Concatenação trocada por consulta parametrizada (`?`)	É a correção da injeção: a entrada é tratada como dado, nunca como SQL.
`validar_email()`	Validação básica: rejeita tipo errado, tamanho excessivo e formato inválido. É uma camada extra e não substitui o `?`.
`print` da query removido	Evita gravar e-mails e tentativas de ataque em logs.
`closing()`	A versão original nunca fechava a conexão.

Como o ataque funcionava: com a entrada `' OR '1'='1`, a query original vira `... WHERE email = '' OR '1'='1'`. A condição é sempre verdadeira e o banco devolve um usuário sem que o atacante conheça nenhum e-mail.

Testes (na pasta `exercicio_9_3`): `python -m unittest -v`. Eles mostram o ataque funcionando na versão original e sendo bloqueado na nova, inclusive com a validação desligada, o que prova que o `?` protege por si só.

Exercício 9.4: LGPD

Checklist *Privacy by Design* para um projeto que coleta **Nome, CPF e Endereço**:

1. **Minimização e finalidade** (art. 6º, I a III): definir por escrito para que serve cada dado e coletar só o necessário. Se o CPF ou o endereço não for indispensável para uma função do sistema, não deve ser pedido.
2. **Base legal e transparência** (arts. 7º a 9º): indicar a base legal de cada tratamento e publicar uma política de privacidade clara. Se a base for o consentimento, usar caixa de aceite desmarcada, registrar data e versão do termo aceito e permitir revogação tão fácil quanto o aceite.
3. **Segurança desde a concepção** (art. 46, § 2º): usar HTTPS, criptografar o CPF no banco, mascarar o CPF em telas e logs (`***.456.789-**`), restringir o acesso por perfil e usar consultas parametrizadas. Ter também um procedimento para comunicar incidentes (art. 48).
4. **Direitos do titular e ciclo de vida** (arts. 15, 16 e 18): oferecer no sistema as funções de consultar, corrigir, exportar e excluir os próprios dados, definir prazo de retenção e eliminar ou anonimizar o

que não for mais necessário.

Observação: Nome, CPF e Endereço são dados **pessoais**, mas não dados pessoais **sensíveis**. Os sensíveis estão listados no art. 5º, II, como dados de saúde, biometria e religião. O portal da clínica do 9.1 trata dados sensíveis; este projeto, com esses três campos, não.

Para revisar

OWASP Top 10:2025

Código	Categoria
A01	Broken Access Control
A02	Security Misconfiguration
A03	Software Supply Chain Failures
A04	Cryptographic Failures
A05	Injection
A06	Insecure Design
A07	Authentication Failures
A08	Software or Data Integrity Failures
A09	Security Logging and Alerting Failures
A10	Mishandling of Exceptional Conditions

Ideias-chave

- **Autenticação** responde "quem é você?"; **autorização** responde "você pode fazer isso?".
- **Tríade CIA:** quem pode ver (confidencialidade), quem pode mudar (integridade), se está no ar quando precisa (disponibilidade).
- **Injeção** acontece quando dados entram misturados com comandos. A correção principal é separar os dois (consulta parametrizada); validar a entrada é uma defesa adicional.
- **Segredos** não ficam no código: use variáveis de ambiente ou cofre.
- **Privacy by Design:** privacidade planejada desde o início e ativa por padrão, não adicionada no fim.